# Toward Fortran Level AMICA in Julia

## Midterm Presentation

**Valentin Morlock, 20.2.2025**

# Agenda

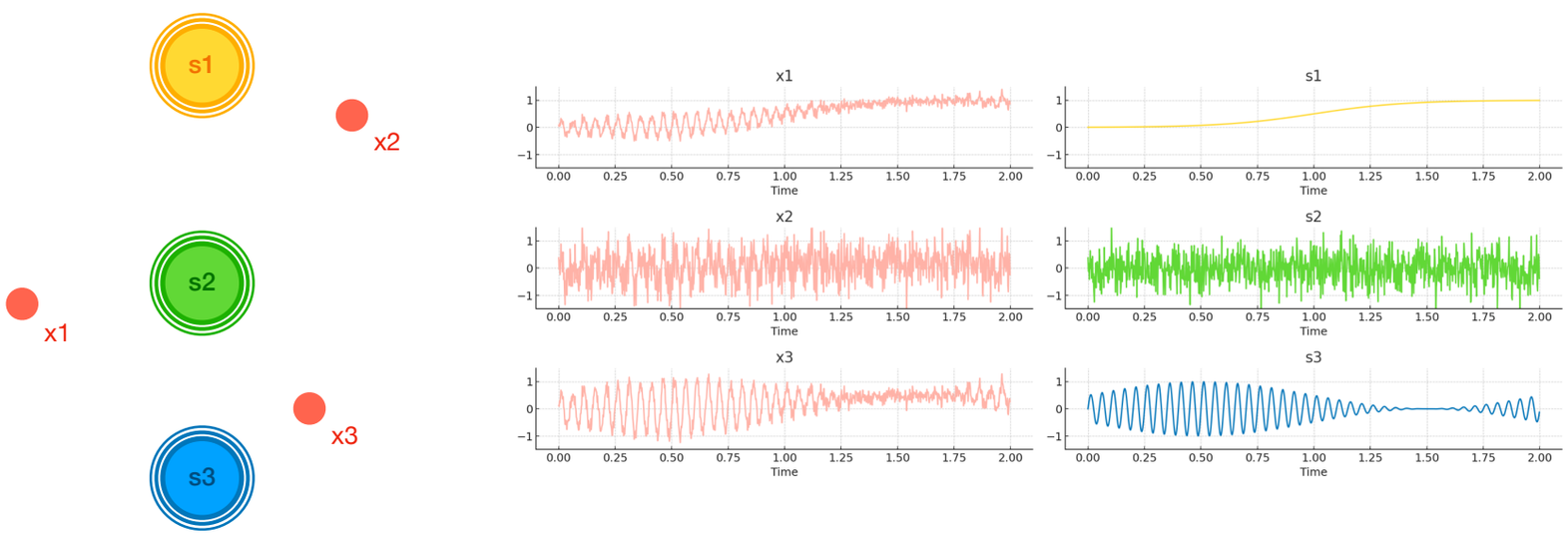
- Recap: ICA, Amica, Research Question
- Status Quo
- Approach
  - Testing
  - Initial Cleanup
  - GPU Support
  - Blockwise Processing
- Preliminary Results
- Next Steps

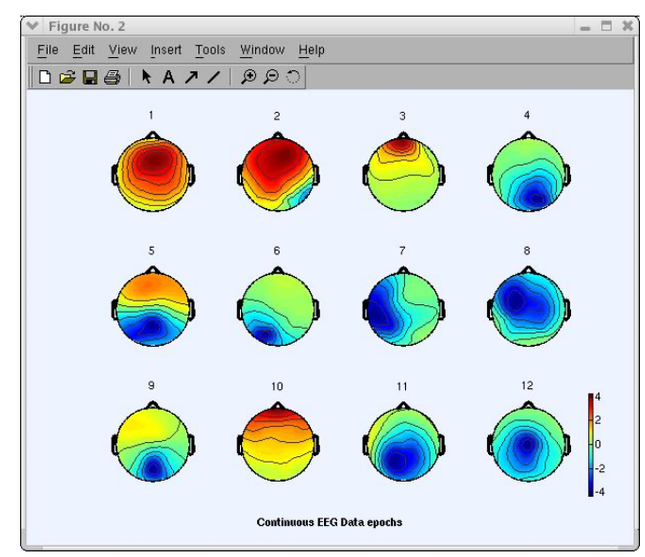
# Independent Component Analysis

- ICA separates mixed signals into original sources by linear transformation from source signals $s$ to mixtures $x$
  $x = A * s$ → estimate $W = A^{-1}$ to reverse mixing $s = x * W$

- Approached e.g. by maximizing entropy or maximizing likelihood

- Used to remove Artifacts (e.g. eye blinks, heartbeats) or separate brain sources from EEG signals

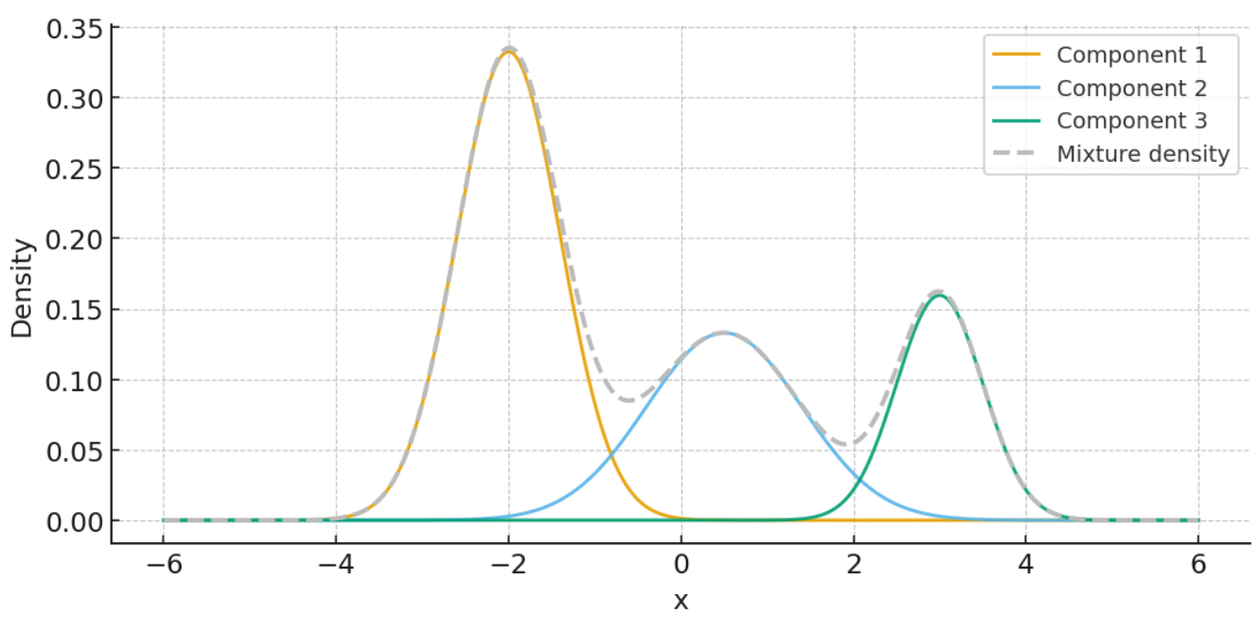


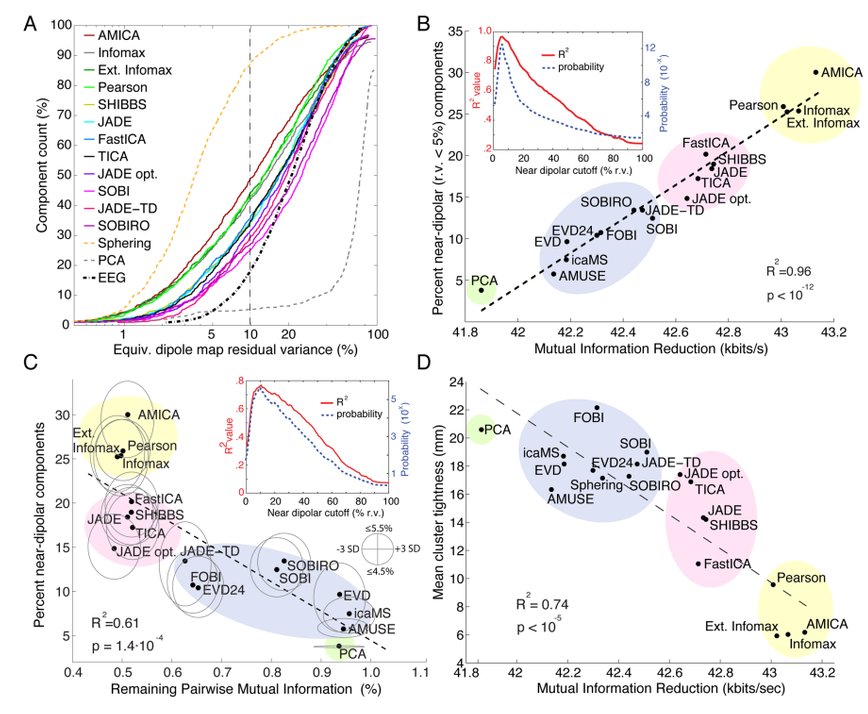




https://eeglab.org/tutorials/06_RejectArtifacts/RunICA.html

# AMICA

- Maximum-likelihood based ICA
- Source densities modeled as Mixtures of Generalized Gaussians
- Multiple models for non-stationarity
- Found to be the best performing ICA algorithm for separating brain signals from non brain source signals




Delorme, Arnaud, et al. "Independent EEG sources are dipolar." *PloS one* 7.2 (2012): e30135.

# Problem & Research Question

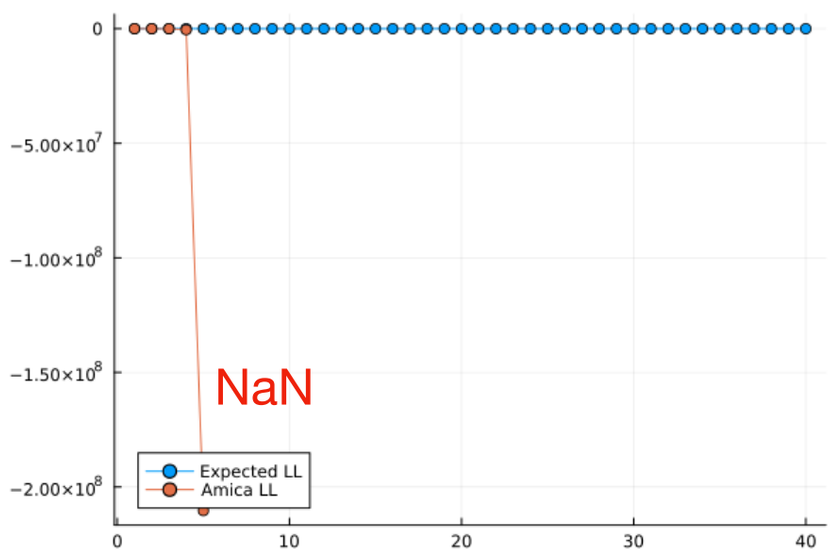
- Three implementations with distinct problems

| | Maintainability | Convergence | Performance |
|---|---|---|---|
| **Fortran** | -- | ++ | ++ |
| **Matlab** | + | - | -- |
| **Julia** | ++ | - | -- |

- Can an optimized Julia AMICA match Fortran in convergence and runtime efficiency?
- Which improvements (e.g. blockwise learning, GPU acceleration) show the strongest contribution?

# Status Quo

- Iteration time roughly matches single-threaded Fortran algorithm, but no parallelization
- High memory consumption
- Results were tested against Matlab but not Fortran
- Issues with numerical stability




| Max. Memory GB | s / Iter |
|---|---|
| CPU ~ 7.5 | ~ 6,0 |

**Dataset for all upcoming benchmarks:**

Memorize.fdt
86,53MB Float32, therefore ~172MB Float64

*(71 x 319500) 71 channel Sternberg dataset from Onton and Makeig 2006*
https://sccn.ucsd.edu/~jason/amica15/Memorize.fdt

# Testing

## Timings and Allocations

*TimerOutputs.jl*

```
                                          Time                   Allocations
                      Tot / % measured:     121s /   6.8%          8.65GiB /  10.2%

Section                       ncalls     time    %tot     avg     alloc    %tot      avg
sphering                           3    467ms    5.6%   156ms    175MiB   19.4%  58.2MiB
update_parameters                  3    2.82s   34.1%   941ms    174MiB   19.3%  58.0MiB
  kernel                           3    2.82s   34.1%   941ms    174MiB   19.3%  58.0MiB
    dA                             3    421ms    5.1%   140ms    174MiB   19.3%  58.0MiB
    dmu_denom                      3    220ms    2.7%  73.3ms   5.86KiB    0.0%  1.95KiB
    dbeta_denom                    3    227ms    2.7%  75.8ms   3.16KiB    0.0%  1.05KiB
    kp                             3    287ms    3.5%  95.6ms   3.16KiB    0.0%  1.05KiB
    dlambda_numer                  3    268ms    3.2%  89.3ms   3.05KiB    0.0%  1.02KiB
    dmu_numer                      3    239ms    2.9%  79.8ms   3.02KiB    0.0%  1.01KiB
    drho_numer                     3    1.13s   13.6%   376ms   2.88KiB    0.0%     981B
    sum_z                          3   32.6ms    0.4%  10.9ms   2.88KiB    0.0%     981B
  prop                             3   33.7μs    0.0%  11.2μs   5.72KiB    0.0%  1.91KiB
  para                             3   32.0μs    0.0%  10.7μs   4.06KiB    0.0%  1.35KiB
  shape                            3   36.8μs    0.0%  12.3μs     32.0B    0.0%    10.7B
  loc                              3   8.32μs    0.0%  2.77μs     32.0B    0.0%    10.7B
  scale                            3   5.70μs    0.0%  1.90μs     32.0B    0.0%    10.7B
update_sources                     3   67.9ms    0.8%  22.6ms    174MiB   19.3%  58.0MiB
removeMean                         3    523ms    6.3%   174ms    174MiB   19.3%  58.0MiB
movetogpu                          3    160ms    1.9%  53.3ms    173MiB   19.2%  57.7MiB
calculate_u_and_Lt                 3    2.50s   30.2%   833ms   17.4MiB    1.9%  5.79MiB
  kernel                           3    2.40s   29.0%   799ms   17.3MiB    1.9%  5.76MiB
  ldet                             3   1.00ms    0.0%   335μs   94.2KiB    0.0%  31.4KiB
  qconst                           3   22.9μs    0.0%  7.62μs   2.88KiB    0.0%     981B
  Q                                3   98.8ms    1.2%  32.9ms      176B    0.0%    58.7B
  sum                              3    137μs    0.0%  45.6μs      176B    0.0%    58.7B
```

```julia
@timeit to "initialize_shape_parameter!" initialize_shape_parameter!(myAmica, lrate)

@timeit to "removeMean" if remove_mean
    removed_mean = removeMean!(data)
end

@timeit to "sphering" if do_sphering
    S = sphering!(data)
    myAmica.S = S
    myAmica.LLdetS = logabsdet(S ▷ Array)[1]
else
    myAmica.S = I
    myAmica.LLdetS = 0
end
```

https://github.com/KristofferC/TimerOutputs.jl

# Testing

## Outputs

- Save intermediary values to bin files in Fortran
- Automatically compile and run Fortran and Julia algorithms
- Compare output and intermediaries

→ Test failures can be narrowed down to individual calculations

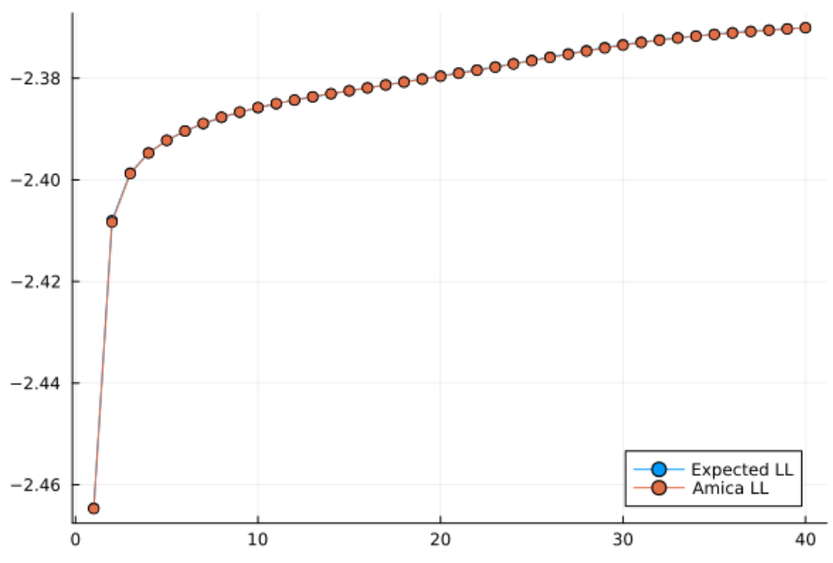
```
compare without newton: Test Failed at /home/fapra_morlock/hallo/Amica.jl/test/integration_tests/integration_test.jl:94
  Expression: myAmica.proportions ≈ alpha
   Evaluated: [0.3720550818120806 0.4629496027629765 0.36499531542494296; 0.3448299353736988 0.4805546759150126 0.37461538871128847; … ;
064445] ≈ [0.31004590151006944 0.38579133563581597 0.30416276285411736; 0.28735827947808124 0.40046222992918745 0.31217949059273875; … ;
6720367]

Stacktrace:
 [1] top-level scope
   @ ~/hallo/Amica.jl/test/integration_tests/integration_test.jl:31
 [2] macro expansion
   @ /scratch/users/morlock/.julia/juliaup/julia-1.12.3+0.x64.linux.gnu/share/julia/stdlib/v1.12/Test/src/Test.jl:1776 [inlined]
 [3] macro expansion
   @ ~/hallo/Amica.jl/test/integration_tests/integration_test.jl:94 [inlined]
 [4] macro expansion
   @ /scratch/users/morlock/.julia/juliaup/julia-1.12.3+0.x64.linux.gnu/share/julia/stdlib/v1.12/Test/src/Test.jl:680 [inlined]
Test Summary:          | Pass  Fail  Total   Time
compare without newton |   11     1     12  23.2s
```

# Initial Cleanup

## Fixing the Julia Implementation

- Adjust to algorithmic changes, e.g. $y^{\rho-1}$ instead of $y^{\rho}$, logsumexp to compute softmax
- Improve numeric stability
- Claude was good at extracting details from Fortran codebase




| Max. Memory GB | | s / Iter |
|---|---|---|
| CPU | ~ 7,5 | ~ 6,0 |
| CPU | ~ 3,5 | ~ 7,1 |

# GPU Support

## Broadcasts vs Kernels

```
# custom kernel
@kernel function compute_kp_atomic!(@Const(z), @Const(y_rho), @Const(y), @Const(shape), output)
    i, j, k = @index(Global, NTuple)
    fp = y_rho[i, j, k] * sign(y[i, j, k]) * shape[j, k]
    zfp_squared = z[i, j, k] * fp * fp / max(z[i, j, k], eps(eltype(z)))
    Atomix.@atomic output[j, k] += zfp_squared
end
```

- Re-use intermediaries without allocation
- Summing is hard (lock congestion, numerical stability)
- CPU performance

```
# broadcast
myAmica.scratch .= (myAmica.z .* (myAmica.y_rho .* sign.(myAmica.y) .* push_dimension(myAmica.shape)))
kp = sum(myAmica.z .* (myAmica.y_rho .* sign.(myAmica.y) .* push_dimension(myAmica.shape)) .^ 2, dims=1)[1, :, :]
```

- Efficient on CPU and GPU
- Unwanted allocations (e.g. sum, re-use calculations), partially solved using scratch arrays

# GPU Support

## Initial Solution

- Mostly broadcasts, two custom kernels
- SingleModelAmica is generic over the kind of array
- Preparing functions run on CPU

```julia
myAmica = fit(Float64,
    SingleModelAmica,
    data;
    maxiter=5,
    Array=CuArray)

…
mutable struct SingleModelAmica{
    T,
    Array1<:DenseArray{T,1},
    Array2<:DenseArray{T,2},
    Array3<:DenseArray{T,3}
} <: AbstractAmica
    source_signals::Array2

    proportions::Array2
    scale::Array2
    location::Array2
    shape::Array2
```

| | Max. Memory GB | s / Iter |
|---|---|---|
| CPU | ~ 7,5 | ~ 6,0 |
| CPU | ~ 3,5 | ~ 7,1 |
| GPU | ~ 15,6 | ~ 0,1 |
| CPU | ~ 3,6 | ~ 6,6 |

# Blockwise Processing

- Idea: most outputs are sums along samples,
  compute the intermediary in blocks and add to the output value

```
# without blocks
scratch ⋅= (z .* (y_rho .* sign.(y) .* shape))
dmu_numer ⋅= sum(scratch, dims=1)[1, :, :]

# with blocks
dmu_numer ⋅= 0.0
for block in 1:num_blocks
    range = ((block-1)*block_size+1):(block*block_size)

    scratch ⋅= (z[range] .* (y_rho[range] .* sign.(y[range]) .* shape[range]))
    dmu_numer .+= sum(scratch, dims=1)[1, :, :]
end
```

- Trivial to implement parallel processing
  by distributing blocks across threads

| Max. Memory GB | | s / Iter |
|---|---|---|
| CPU | ~ 7,5 | ~ 6,0 |
| CPU | ~ 3,5 | ~ 7,1 |
| GPU | ~ 15,6 | ~ 0,1 |
| CPU | ~ 3,6 | ~ 6,6 |
| GPU | ~ 1,0 | ~ 0,3 |
| CPU | ~ 0,9 | ~ 4,8 |
| CPU*64 | ~ 1,9 | ~ 0.5 |

Blocksize: 1000

# Preliminary Results

| Implementation | Acceleration | Datatype | Threads | Blocksize | Memory (GB) | s / Iter |
|---|---|---|---|---|---|---|
| **Fortran** | MKL | Float64 | 1 | 319500 | 3,49 | 4,45 |
| **Fortran** | MKL | Float64 | 1 | 1000 | 0,22 | 3,81 |
| **Fortran** | MKL | Float64 | 64 | 1000 | 0,99 | 0,16 |
| **Julia** | - | Float64 | 1 | 319500 | 5,30 | 5,59 |
| **Julia** | - | Float64 | 1 | 1000 | 0,99 | 4,82 |
| **Julia** | - | Float64 | 64 | 1000 | 1,89 | 0,42 |
| **Julia** | CUDA | Float64 | 1 | 319500 | 5,8 | 0,11 |
| **Julia** | CUDA | Float64 | 1 | 1000 | 1,07 | 0,45 |
| **Julia** | - | Float32 | 1 | 319500 | 2,78 | 4,71 |
| **Julia** | - | Float32 | 1 | 1000 | 0,79 | 4,46 |
| **Julia** | - | Float32 | 64 | 1000 | 1,21 | 0,28 |
| **Julia** | CUDA | Float32 | 1 | 319500 | 3,30 | 0,046 |
| **Julia** | CUDA | Float32 | 1 | 1000 | 0,90 | 0,36 |

Average of 40 iterations, measured from iteration 2 for Julia, newton method active for all iterations

# Next Steps

- Adapt testing suite to blockwise processing (~ 1 week)
- Improve code structure and docs (~ 1 week)
- Write thesis (~ 4 weeks)